

Chapter 2: Tree

2.2 Definition of Tree

A tree can be defined in several ways. One natural way to define a tree is recursively.

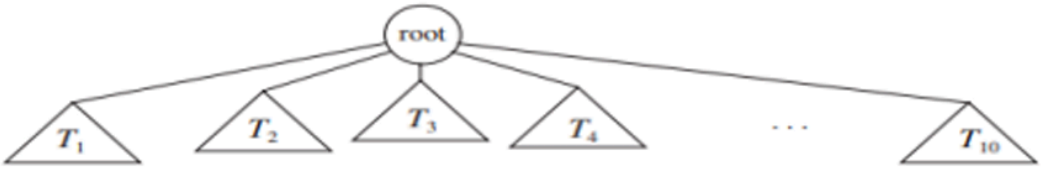


Figure 2.1: Generic Tree

Recursive Definition

A tree is a collection of nodes. The collection can be empty; otherwise, a tree consists of:

- A distinguished node, **r**, called the **root**
- Zero or more nonempty (sub)trees $T_1, T_2, \ldots, T_k$, each of whose roots are connected by a directed edge from **r**

The root of each subtree is said to be a **child** of **r**, and **r** is the **parent** of each subtree root.

General Description

Tree data structure is a specialized data structure to store data in a hierarchical manner. It is used to organize and store data in the computer to be used more effectively. It consists of:

- A central node (root)
- Structural nodes
- Sub-nodes

All connected via edges. We can also say that tree data structure has roots, branches, and leaves connected.

Tree data structure is a hierarchical structure that is used to represent and organize data in a way that is easy to navigate and search. It is a collection of nodes that are connected by edges and has a hierarchical relationship between the nodes.

Key characteristics:

- The topmost node of the tree is called the **root**
- The nodes below it are called the **child nodes**
- Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure
- The data in a tree are **not stored in a sequential manner** (i.e., they are not stored linearly)
- Instead, they are arranged on multiple levels (hierarchical structure)
- For this reason, the tree is considered to be a **nonlinear data structure**

Basic Terminologies in Tree Data Structure

- **Parent Node**: The node which is a predecessor of a node is called the parent node of that node. Example: {B} is the parent node of {D,E}.
- **Child Node**: The node which is the immediate successor of a node is called the child node of that node. Examples: {D,E} are the child nodes of {B}.
- **Root Node**: The topmost node of a tree, i.e., the node that does not have any parent, is called the root node. Example: {A} is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to every other node.
- **Leaf (External) Node**: Nodes that do not have any children are called leaf nodes. Example: {I,J,K,F,G,H} are the leaf nodes of the tree.

- **Ancestor of a Node**: Any predecessor nodes on the path from the root to that node are called the ancestors of that node. Example: {A,B} are the ancestor nodes of {E}.
- **Descendant**: A node **x** is a descendant of another node **y** if and only if **y** is an ancestor of **x**.
- **Sibling**: Children of the same parent are called siblings. Example: {D,E} are siblings.
- **Level of a Node**: The number of edges on the path from the root node to that node. The root has level 0.
- **Internal Node**: A node with at least one child is called an internal node.
- **Neighbor of a Node**: The parent or children of a node are the neighbors of that node.
- **Subtree**: Any node together with all of its descendants.

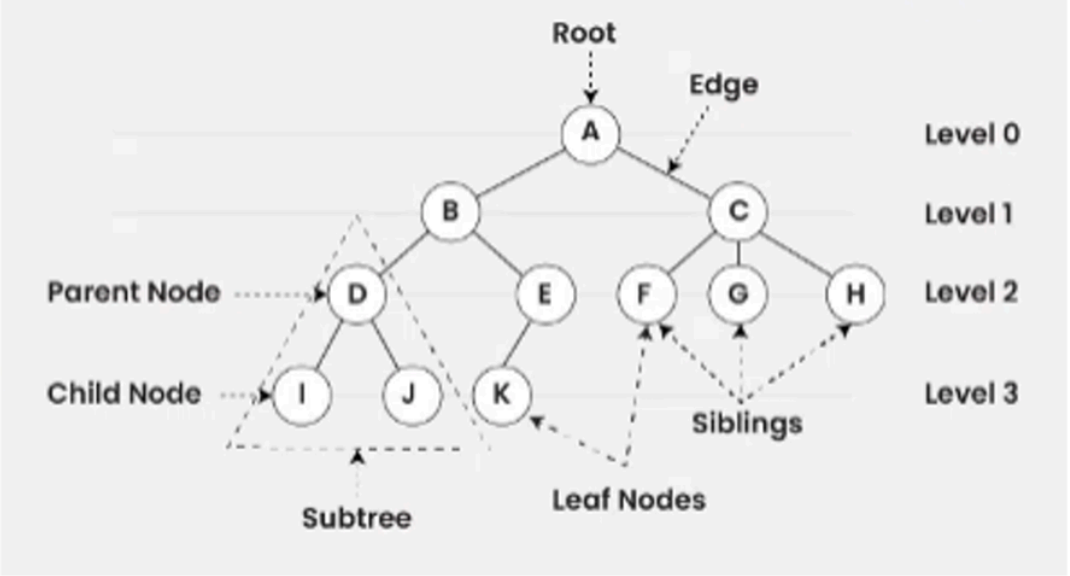


Figure 2.2: tree data structure

Tree Properties from Recursive Definition

From the recursive definition, we find that a tree is a collection of **N nodes**, one of which is the root, and **N−1 edges**. The fact that there are N−1 edges follows from the observation that each edge connects some node to its parent, and every node except the root has exactly one parent.

Example observations:

- In the tree, the root is A

- Node F has A as a parent and K, L, and M as children
- Each node may have an arbitrary number of children, possibly zero
- Nodes with no children are known as **leaves**
- The leaves in the tree above are: B, C, H, I, P, Q, K, L, M, and N
- Nodes with the same parent are **siblings**; thus, K, L, and M are all siblings
- **Grandparent** and **grandchild** relations can be defined in a similar manner

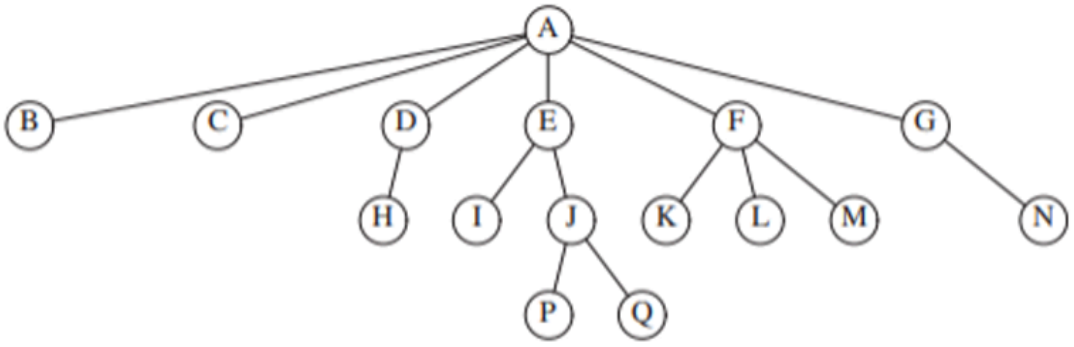


Figure 2.3: A tree

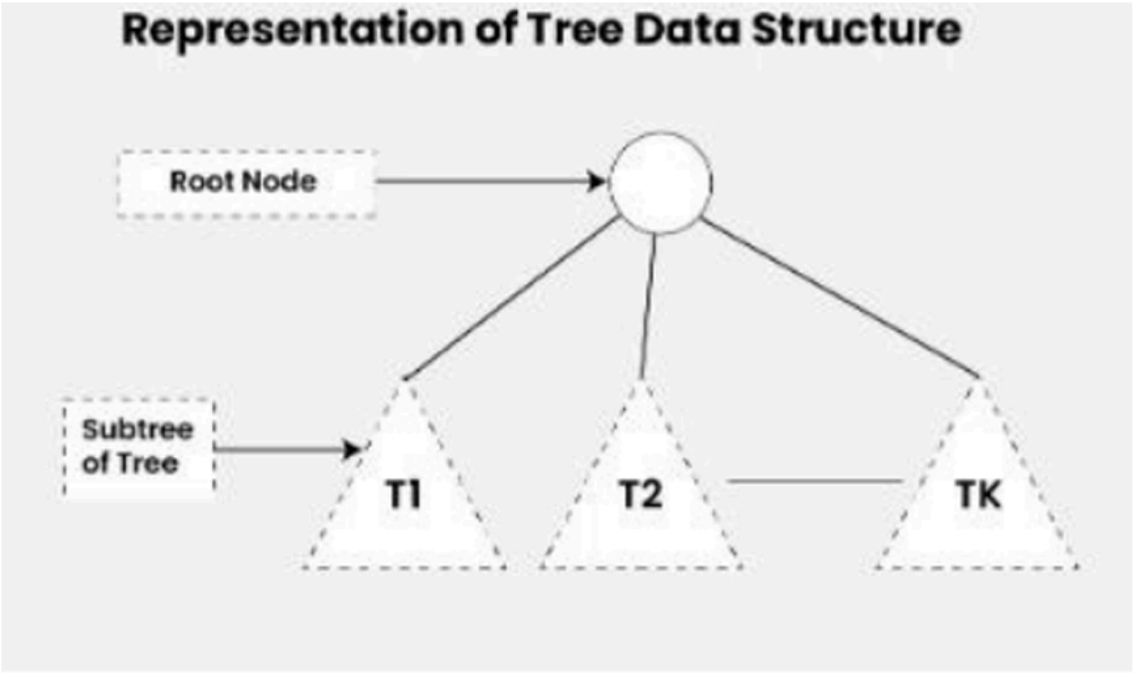


Figure 2.4: Representation of Tree Data Structure

2.3 Representation of Tree Data Structure

A tree consists of a root node, and zero or more subtrees $T_1, T_2, ..., T_k$ such that there is an edge from the root node of the tree to the root node of each subtree. The subtree of a node **X** consists of all the nodes that have node **X** as the ancestor node.

Representation of a Node in Tree Data Structure

A tree can be represented using a collection of nodes. Each of the nodes can be represented with the help of class or structs. Below is the representation of Node in C++ language:

```
class Node {
public:
    int data;
    Node* first_child;
    Node* second_child;
    Node* third_child;
    // ...
    Node* nth_child;
};
```

Importance of Tree Data Structure

1. **Hierarchical Information**: One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer.
2. **Moderate Access/Search**: Trees (with some ordering, e.g., BST) provide moderate access/search (quicker than linked lists and slower than arrays).
3. **Moderate Insertion/Deletion**: Trees provide moderate insertion/deletion (quicker than arrays and slower than unordered linked lists).
4. **No Upper Limit on Nodes**: Like linked lists and unlike arrays, trees don't have an upper limit on the number of nodes, as nodes are linked using pointers.

Types of Tree Data Structures

Tree data structure can be classified into three types based upon the number of children each node of the tree can have:

- **Binary Tree**: In a binary tree, each node can have a maximum of two children linked to it. Some common types of binary trees include full binary trees, complete binary trees, balanced binary trees, and degenerate or pathological binary trees.

- **Ternary Tree**: A Ternary Tree is a tree data structure in which each node has at most three child nodes, usually distinguished as "left", "mid" and "right".
- **N-ary Tree or Generic Tree**: Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children (duplicate references are not allowed). Unlike the linked list, each node stores the address of multiple nodes.

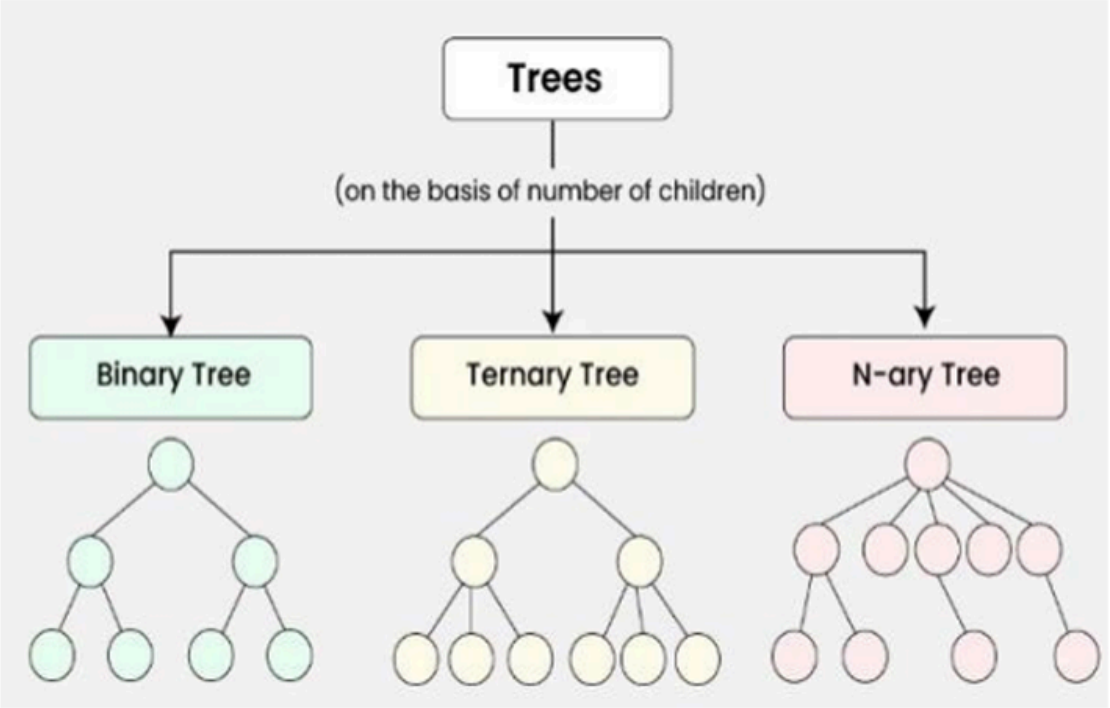


Figure 2.6: Types of Tree Data Structures:

Basic Operations of Tree Data Structure

- **Create**: Create a tree in the data structure.
- **Insert**: Inserts data in a tree.
- **Search**: Searches specific data in a tree to check whether it is present or not.
- **Traversal**:
  - Depth-First-Search Traversal
  - Breadth-First-Search Traversal

Properties of Tree Data Structure

- **Number of Edges**: If a tree has N nodes, then it has N−1 edges. In an (undirected) tree there is exactly one simple path between any two nodes.

- **Depth of a Node**: The number of edges on the path from the root to that node (typically depth(root) = 0).
- **Height of a Node**: The length of the longest downward path from the node to a leaf (thus leaves have height 0).
- **Height of the Tree**: The length of the longest path from the root to any leaf (i.e., the maximum depth).
- **Degree of a Node**: The number of children of the node. A leaf has degree 0. The degree of a tree is the maximum node degree in the tree.

Additional Terminology

Path: A sequence of edges. A path from node **u** to node **v** is a sequence of nodes **$(n_1, n_2, \ldots, n_k)$** with **$n_1 = u$**, **$n_k = v$**, and each consecutive pair connected by an edge. The length of the path is the number of edges, **$k-1$**. There is a path of length zero from any node to itself.

- In a rooted (directed-downward) view, there is a unique path from the root to every node
- In an undirected view of the tree, there is exactly one simple path between any two nodes

Key measurements:

- **Length of a path**: Number of edges on the path
- **Depth**: Length of the unique path from the root to that node

For any node **n_i**:

- The **depth** of **n_i** is the length of the unique path from the root to **n_i**. Thus, the root is at depth 0.
- The **height** of **n_i** is the length of the longest path from **n_i** to a leaf. Thus all leaves are at height 0.
- The **height of a tree** is equal to the height of the root.

Example: For the tree in Figure 2.3:

- E is at depth 1 and height 2
- F is at depth 1 and height 1
- The height of the tree is 3

Additional notes:

- The depth of a tree is equal to the depth of the deepest leaf; this is always equal to the height of the tree
- If there is a path from $\mathbf{n_1}$ to $\mathbf{n_2}$, then $\mathbf{n_1}$ is an ancestor of $\mathbf{n_2}$ and $\mathbf{n_2}$ is a descendant of $\mathbf{n_1}$
- If $\mathbf{n_1 \neq n_2}$, then $\mathbf{n_1}$ is a **proper ancestor** of $\mathbf{n_2}$ and $\mathbf{n_2}$ is a **proper descendant** of $\mathbf{n_1}$

Applications of Tree Data Structure

- **File System**: This allows for efficient navigation and organization of files.
- **Data Compression**: Huffman coding is a popular technique for data compression that involves constructing a binary tree where the leaves represent characters and their frequency of occurrence. The resulting tree is used to encode the data in a way that minimizes the amount of storage required.
- **Compiler Design**: In compiler design, a syntax tree is used to represent the structure of a program.
- **Database Indexing**: B-trees and other tree structures are used in database indexing to efficiently search for and retrieve data.
- **Expression Tree**: Expression tree for (a + b ∗ c) + ((d ∗ e + f) ∗ g).

Advantages of Tree Data Structure

- Trees offer efficient searching depending on the type of tree, with average search times of O(log n) for balanced trees like AVL.
- Trees provide a hierarchical representation of data, making it easy to organize and navigate large amounts of information.
- The recursive nature of trees makes them easy to traverse and manipulate using recursive algorithms.

Disadvantages of Tree Data Structure

- Unbalanced trees, meaning that the height of the tree is skewed toward one side, can lead to inefficient search times.
- Trees demand more memory space than some other data structures like arrays and linked lists, especially when the tree is very large.
- The implementation and manipulation of trees can be complex and require a good understanding of the algorithms.

2.4 Binary Trees

Definition

A binary tree **T** is either empty or has these properties:

- Has a root node
- Has two sets of nodes: left subtree **LT** and right subtree **RT**
- **LT** and **RT** are binary trees

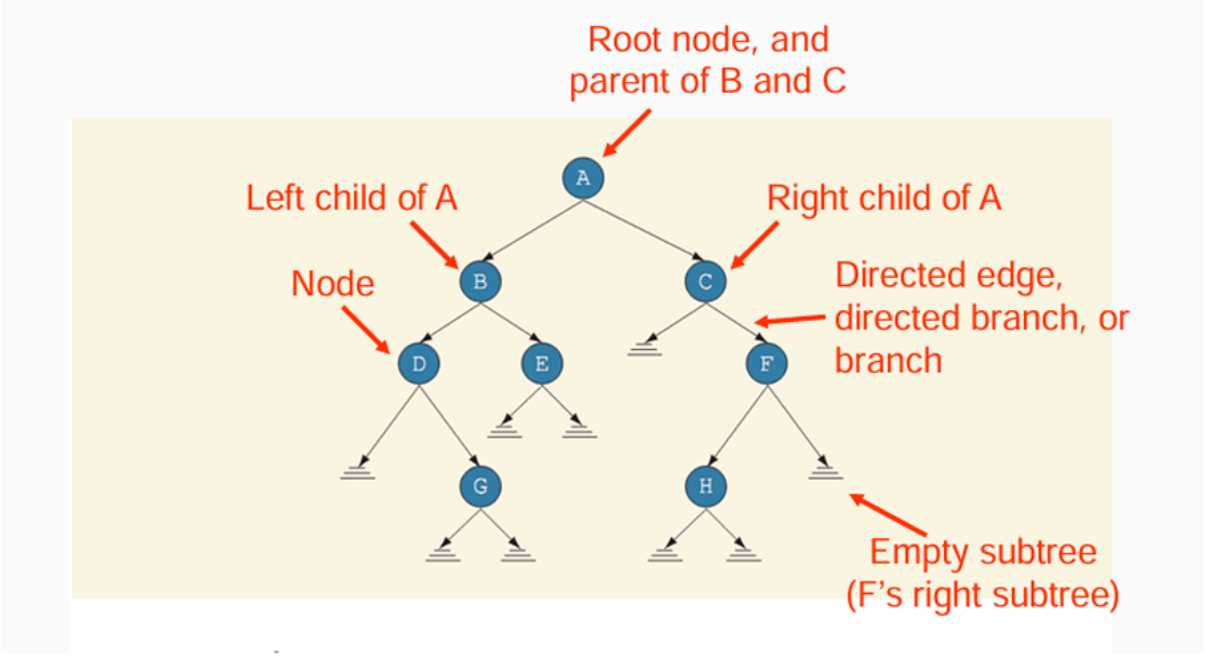


Figure 2.7: Binary Tree traversals

Binary Tree Node Characteristics

- Every node has at most two children
- A node:
  - Stores its own information
  - Keeps track of its left subtree and right subtree using pointers (lLink and rLink pointers)
- A pointer to the root node of the binary tree is stored outside the tree in a pointer variable

Additional definitions:

- **Leaf**: Node that has no left and right children
- **Parent**: U is parent of V if there is a branch from U to V
- **Path**: There is a unique path from the root to every node

- **Length of a path**: Number of branches (edges) on the path
- **Level of a node**: Number of branches on the path from the root to the node (the root node has level 0)
- **Height of a binary tree**: Number of nodes on the longest path from the root to a leaf

Binary Tree Node Structure

A binary-tree node stores a data value and two child pointers. If a child does not exist, the corresponding pointer is set to nullptr. Thus each node conceptually has three fields:

- A **Value** field that holds the data
- Two pointer fields: **Left** and **Right**, which refer to the left and right subtrees, respectively

```
struct Node {
    int data;
    struct Node *left, *right;
};
```

In C++, a minimal node definition uses a struct with three members: an integer data field and two pointers to Node for the left and right children. To create a node at runtime, we allocate it on the heap, assign the value, and initialize both child pointers to nullptr.

The following function creates a new tree node, assigns its data value, and initializes both child pointers to NULL:

```
Node* newNode(int data) {
    Node* temp = new Node;
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}
```

2.4.1 Binary Search Tree Operations

- **Contains(value)**: Returns true if "value" exists in the tree.
- **Insert(value)**: Adds a value to the tree.
- **Traverse**: (Display) Displays all nodes in the tree.

- **Remove(value)**: Deletes a node from the tree.

FIND OPERATION

1. Start at the root node as current node.
2. While the node is not found and the tree did not end:
    - If the value is less than the data in the current node: then search in the left subtree.
    - Else if the value is greater than the data in the current node: then search in the right subtree.
    - Else: then the node is found.
3. End while.

INSERT OPERATION

1. Start at root node as current node.
2. While the end of the tree is not reached:
    - If new node's data < current's data: then search left.
    - Else if new node's key > current's key: then search right.
    - Else: then it already exists (stop).
3. Add the node in its correct place (left or right of the targeted subtree).
4. Increment the size.

Insertion function definition:

```
BST* BST::Insert(BST* root, int value) {
    if (!root) {
        // Insert the first node, if root is NULL.
        return new BST(value);
    }
    // Insert data.
    if (value > root->data) {
        // Insert right node data, if the 'value'
        // to be inserted is greater than 'root' node data.
        // Process right nodes.
        root->right = Insert(root->right, value);
    } else {
```

```
        // Insert left node data, if the 'value'
        // to be inserted is greater than 'root' node data.
        // Process left nodes.
        root->left = Insert(root->left, value);
    }
    // Return 'root' node, after insertion.
    return root;
}
```

Exercise: Draw the Binary Search Tree resulting from inserting the following values in the same order: 100, 200, 90, 150, 125, 88, 99, 210

Tree Traversal

A tree traversal is a specific order in which to trace the nodes of a tree.

Three Common Methods

1. **In-order**: left, root, right
2. **Pre-order**: root, left, right
3. **Post-order**: left, right, root

1. In-order Traversal (left, root, right)

The in-order traversal gives: 16, 25, 41, 46, 53, 55, 63, 64, 65, 70, 74.

Important: For a Binary Search Tree, an in-order traversal outputs the keys in non-decreasing (sorted) order.

```
/* Given a binary tree, print its nodes in inorder */
void printInorder(struct Node* node) {
    if (node == NULL)
        return;
    /* first recur on left child */
    printInorder(node->left);
    /* then print the data of node */
    cout << node->data << " ";
    /* now recur on right child */
    printInorder(node->right);
}
```

2. Pre-order Traversal (root, left, right)

Visit the current node first, then recursively traverse the left subtree, then the right subtree. This order is useful for serializing a tree and for prefix (Polish) notation of expression trees.

Characteristics:

- **Idea**: Process the node, then recurse left, then right
- **Time**: O(n) over n nodes
- **Space**: O(h) due to recursion stack, where h is the height
- **Base case**: If the node is NULL, return
- **Note**: Unlike in-order on a BST, pre-order does not produce sorted output

```
/* Given a binary tree, print its nodes in preorder */
void printPreorder(struct Node* node) {
    if (node == NULL)
        return;
    /* first print data of node */
    cout << node->data << " ";
    /* then recur on left subtree */
    printPreorder(node->left);
    /* now recur on right subtree */
    printPreorder(node->right);
}
```

3. Post-order Traversal (left, right, root)

```
/* Given a binary tree, print its nodes in postorder */
void printPostorder(struct Node* node) {
    if (node == NULL)
        return;
    /* first recur on left child */
    printPostorder(node->left);
    /* now recur on right child */
    printPostorder(node->right);
    /* then print data of node */
    cout << node->data << " ";
}
```

Driver Program Example

```
/* Driver program to test above functions*/
int main() {
    struct Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);

    cout << "\nPreorder traversal of binary tree is \n";
    printPreorder(root);

    cout << "\nInorder traversal of binary tree is \n";
    printInorder(root);

    cout << "\nPostorder traversal of binary tree is \n";
    printPostorder(root);

    return 0;
}
```

---

4. Remove(value): Deletes a Node from the Tree

To delete a key from a binary search tree while preserving the BST invariant (left subtree < node < right subtree), consider three cases:

Case 1: Leaf Node (No Children)

Detach it from its parent by setting the corresponding child pointer to nullptr, then free it. If it is the root, the tree becomes empty.

Steps:

- Find the parent of the target leaf (e.g., parent of 42 is 46)
- Set the corresponding child pointer of the parent to nullptr
- Delete the leaf node

Case 2: One Child

Promote the single child to the node's position by linking the parent directly to that child; then delete the node. If the node is the root, the child becomes the new root.

Steps:

- Locate the node to delete (e.g., 16) and its parent (e.g., 41)
- Since 16 has exactly one child (25), link the parent directly to that child
- Delete the node 16

Case 3: Two Children

Replace the node's key with either:

- The **in-order successor** (minimum of the right subtree), or
- The **in-order predecessor** (maximum of the left subtree)

Then delete that successor/predecessor from its original location (which becomes Case 1 or Case 2).

Steps:

- Target node has two children (e.g., 41)
- Go to its right subtree and take the leftmost node (the in-order successor)
- Here, the successor is 42
- Replace node 41 with 42
- Remove the original successor node from its position

Cost Analysis

- **Time**: O(h) where h is the height (O(n) worst case, O(log n) in balanced trees)
- **Space**: O(1) (iterative) or O(h) (recursive)

Important Notes

- Decide a duplicates policy and handle the root as a special case
- In manual-memory settings, free the removed node

---

5. Helper Function: Find Parent

Returns the parent of the given value.

```
// C function to search a given key in a given BST
struct node* search(struct node* root, int key) {
    // Base cases: root is null or key is present at root
    if (root == NULL || root->key == key)
```

```
        return root;

    // Key is greater than root's key
    if (root->key < key)
        return search(root->right, key);

    // Key is smaller than root's key
    return search(root->left, key);
}
```

Example: FindParent(46)

- From root 60, go left to 41
- Then right to 53
- The parent of 46 is 53
